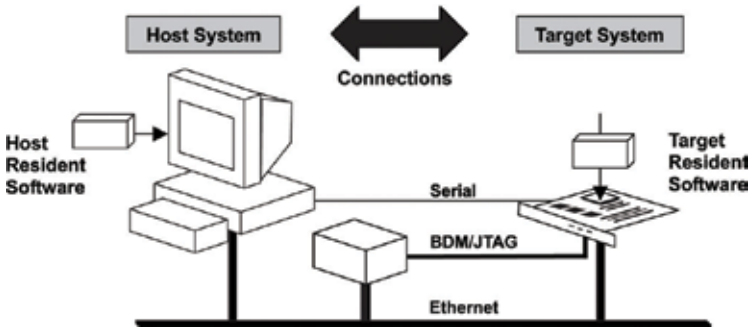




Embedded debugging schematic

Mind well, the real world and real time constraints apply here too, think of it like this, say you stop on a breakpoint where your system is controlling a servo now if you stop at that particular point chances are that you'll permanently damage the motor. Hence, developers extensively use serial IO channels and error-message style debugging in the embedded systems paradigm.

Debugging a microprocessor centric embedded system is different from debugging an embedded system where processing is performed by DSP, FPGA or a co-processor. Today, a large number of embedded systems have started using processors have more than one core. A common problem that developers face is the proper synchronisation of software execution while debugging. In such cases the system will have to implement low-level debugging on the busses between the processor cores. Thus the debugging strategies vary from system to system. Following are some of the methods involving both hardware and software which can be used to debug an embedded system.

Resident Debugger

A resident debugger functions as a normal debugger in our traditional development paradigm. The program runs directly on the microcontroller itself, while the developer is able to control the actions from the host machine. A resident debugger usually occupies some resources from the target machine including a communication port, an interrupt to handle single stepping and some memory for the resident module of the debugger.

External Debuggers

We mentioned earlier that programmers developing in this paradigm rely on serial IO channels and error-message style debugging. This debugger